



Venus Protocol: Fixed Rate Vaults

Security Review

Cantina Managed review by:
Kankodu, Security Researcher
MostafaYassin, Security Researcher

June 1, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	4
3.1	Medium Risk	4
3.1.1	Suppliers can force institution-default failure by pushing fundraising above <code>minBorrowCap</code> near deadline	4
3.1.2	Institutions cannot top up collateral during settlement while liquidation remains enabled	5
3.1.3	ERC-4626 standard violation in <code>deposit</code> and <code>mint</code> due to clamping	7
3.1.4	Defaulting institution can reclaim confiscated margin through supplier shares	7
3.1.5	<code>_baseURI</code> not being overridden returns empty metadata	9
3.2	Low Risk	9
3.2.1	<code>sweep()</code> can transfer core vault assets without token restrictions	9
3.2.2	PSR <code>updateAssetsState</code> failure is handled inconsistently	10
3.2.3	Close-factor rounding can block liquidation of dust debt	10
3.2.4	Inconsistent handling of Fee-on-Transfer tokens	11
3.3	Gas Optimization	12
3.3.1	Dead code: <code>positionTokenId == 0</code> guard can never be triggered	12
3.3.2	Unnecessary address prediction in <code>createVault</code>	12
3.4	Informational	13
3.4.1	NatSpec for <code>openVault</code> references vault state "Open" that doesn't exist	13
3.4.2	NatSpec for <code>_enterLock</code> claims a USD valuation is stored when it is not	13
3.4.3	<code>withdraw</code> and <code>redeem</code> duplicates ERC-4626 Logic	14
3.4.4	Oracle pricing logic is duplicated across multiple functions	14

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Venus Protocol is a leading lending and borrowing platform on the BNB Chain that prioritizes enhanced risk management, decentralization, and user experience.

From May 13th to May 19th the Cantina team conducted a review of [fixed-rate-vaults](#) on commit hash [aafff3e6](#). The team identified a total of **15** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	5	1	4
Low Risk	4	1	3
Gas Optimizations	2	2	0
Informational	4	3	1
Total	15	7	8

2.1 Scope

The security review had the following components in scope for [fixed-rate-vaults](#) on commit hash [aafff3e6](#):

```
src
├── BaseVault.sol
├── institutional-vault
│   ├── InstitutionalLoanVault.sol
│   ├── InstitutionalVaultController.sol
│   ├── InstitutionPositionToken.sol
│   └── LiquidationAdapter.sol
├── interfaces
│   ├── IIInstitutionalLoanVault.sol
│   ├── IIInstitutionalVaultController.sol
│   ├── IIInstitutionalVaultTypes.sol
│   ├── IIInstitutionPositionToken.sol
│   ├── ILiquidationAdapter.sol
│   ├── IProtocolShareReserve.sol
│   ├── IResilientOracle.sol
│   ├── IVaultController.sol
│   └── IVaultTypes.sol
```

3 Findings

3.1 Medium Risk

3.1.1 Suppliers can force institution-default failure by pushing fundraising above `minBorrowCap` near deadline

Severity: Medium Risk

Context: (No context files were provided by the reviewer)

Description: During the `Fundraising` to terminal-state transition, the vault treats a failed fundraise differently depending on whether `totalRaised` reached `minBorrowCap`.

If `totalRaised < minBorrowCap`, the vault simply fails and lenders can withdraw their deposited supply asset. If `totalRaised >= minBorrowCap` but the institution has not deposited `idealCollateralAmount`, the vault marks the institution as defaulted and confiscates the upfront margin for lender compensation.

This creates a griefing/extraction scenario where a supplier can wait until shortly before fundraising ends, deposit enough supply asset to push `totalRaised` above `minBorrowCap`, and force the vault into the institution-default branch if the institution has only deposited margin and has not yet topped up to `idealCollateralAmount`.

Affected Code:

```
if (
    totalRaised >= _config.minBorrowCap
    && _instRuntime.totalCollateralDeposited >= _instConfig.idealCollateralAmount
) {
    _enterLock(totalRaised);
    s = VaultState.Lock;
} else {
    _enterFailed(totalRaised);
    return _runtime.state;
}
```

```
function _enterFailed(
    uint256 totalRaised
) internal {
    _runtime.settlementAmount = totalRaised;
    _stateTransition(VaultState.Failed);
    emit VaultFailed(totalRaised, _config.minBorrowCap);

    if (totalRaised >= _config.minBorrowCap) {
        uint256 marginAmount = (_instConfig.idealCollateralAmount *
            ↪ _instConfig.marginRate) / MANTISSA_ONE;
        _instRuntime.institutionDefaulted = true;
        _instRuntime.confiscatedMarginRemaining = marginAmount;
        emit MarginConfiscated(marginAmount);
    }
}
```

Attack Scenario:

1. A vault enters `Fundraising` after the institution deposits only the upfront margin amount.
2. The institution has not yet deposited the full `idealCollateralAmount`.
3. Near `openEndTime`, `totalRaised` remains below `minBorrowCap`.
4. An attacker deposits enough supply asset to make `totalRaised >= minBorrowCap`.
5. Fundraising ends.
6. The vault does not enter `Lock` because `totalCollateralDeposited < idealCollateralAmount`.

7. `_enterFailed()` marks `institutionDefaulted = true` and sets `confiscatedMarginRemaining = marginAmount`.
8. The attacker withdraws their supply principal and receives a pro-rata share of the confiscated margin compensation.

Impact: The institution can lose its upfront margin even though the vault would otherwise have failed due to insufficient lender demand. A malicious supplier can convert a low-demand fundraise into an institution-default event by strategically depositing near the deadline.

The impact depends on `marginRate` and `idealCollateralAmount`. If the margin is material, the attacker can extract value from the institution while also forcing the vault to fail instead of giving the institution a practical opportunity to top up collateral.

Recommendation: Require the full `idealCollateralAmount` before `openVault()` instead of only requiring upfront margin.

Venus Protocol: Acknowledged. The margin mechanism is intentional — institutes are expected to provide the full `idealCollateralAmount` regardless of total raised. The behavior is designed to apply pressure on institutions to provide collateral, and Venus Protocol will add documentation clarifying that institutions should not rely solely on total raised to decide whether to supply.

Cantina Managed: Acknowledged.

3.1.2 Institutions cannot top up collateral during settlement while liquidation remains enabled

Severity: Medium Risk

Context: (No context files were provided by the reviewer)

Description: After the lock period ends, the vault can enter `PendingSettlement` while debt is still outstanding. In this state, the institution can repay debt, but it cannot deposit additional collateral because `depositCollateral()` is only allowed in `WaitingForMargin`, `Fundraising`, and `Lock`.

At the same time, health-factor liquidation remains enabled in `PendingSettlement` and `SettlementDeadlineExceeded`. If collateral value drops after `lockEndTime`, the institution cannot restore collateralization by topping up collateral during the settlement window. Its only defense is to repay supply debt immediately.

If the position later crosses into true bad debt, `repayBadDebt()` can move the vault to `Liquidated`. Once the vault is `Liquidated`, the institution cannot withdraw remaining collateral through the normal `withdrawCollateral()` path.

Affected Code: Collateral top-ups are disallowed after `Lock`:

```
function depositCollateral(
    uint256 amount
) external onlyPositionHolder nonReentrant whenNotPaused {
    VaultState s = _checkAndAdvanceState();
    if (s != VaultState.WaitingForMargin && s != VaultState.Fundraising && s !=
        VaultState.Lock) {
        revert InvalidState();
    }
    //...
}
```

Normal repayment is allowed in settlement states:

```
if (s != VaultState.Lock && s != VaultState.PendingSettlement && s !=
    VaultState.SettlementDeadlineExceeded) {
    revert InvalidState();
}
```

Health-factor liquidation is also allowed in settlement states:

```
if (s != VaultState.Lock && s != VaultState.PendingSettlement && s !=  
    ↪ VaultState.SettlementDeadlineExceeded) {  
    revert InvalidState();  
}  
  
(, uint256 shortfall) = _getHypotheticalVaultLiquidity(0, 0);  
if (shortfall == 0) revert NotLiquidatable();
```

The bad-debt rescue path terminally marks the vault as `Liquidated` :

```
_receiveRepayment(msg.sender, repayAmount);  
  
if (_runtime.totalDebt > _computeTotalInterest()) revert InsufficientRepayment();  
_stateTransition(VaultState.Liquidated);  
emit VaultLiquidated(IERC20(asset()).balanceOf(address(this)));  
_settleProtocolShare();
```

Institution collateral withdrawal is not available in `Liquidated` :

```
if (s != VaultState.Lock && s != VaultState.Matured && s != VaultState.Failed) {  
    revert InvalidState();  
}
```

Proof of Concept:

1. A vault reaches `Lock` , the institution claims the raised funds, and debt becomes principal plus fixed interest.
2. The lock period ends and the vault enters `PendingSettlement` with non-zero debt.
3. Before the settlement deadline, the collateral price drops and the vault becomes unhealthy under the liquidation-threshold check.
4. The institution has additional collateral available, but `depositCollateral()` reverts because the vault is no longer in `Lock` .
5. The institution must repay supply debt immediately to restore health; it cannot defend the position by adding collateral.
6. Liquidators can liquidate the vault while it is in `PendingSettlement` .
7. If the position deteriorates into true bad debt, `repayBadDebt()` can move the vault to `Liquidated` .
8. Once `Liquidated` , the institution cannot withdraw remaining collateral through normal protocol functions.

Recommendation: Allow the institution to top up collateral during `PendingSettlement` . This preserves the purpose of the settlement window by giving the institution two non-terminal ways to defend the vault before the settlement deadline: repay debt or add collateral.

```
if (  
    s != VaultState.WaitingForMargin &&  
    s != VaultState.Fundraising &&  
    s != VaultState.Lock &&  
    s != VaultState.PendingSettlement  
) {  
    revert InvalidState();  
}
```

Venus Protocol: Acknowledged. During the `PendingSettlement` period, institutions are required to repay the borrowed asset. Allowing collateral top-ups at this stage is intentionally excluded — the settlement window is designed to increasingly pressure institutions to repay, and adding collateral would reduce that pressure.

Cantina Managed: Acknowledged.

3.1.3 ERC-4626 standard violation in `deposit` and `mint` due to clamping

Severity: Medium Risk

Context: (No context files were provided by the reviewer)

Description: The ERC-4626 standard mandates specific invariants for `deposit` and `mint` :

- `deposit/assets, receiver` must deposit *exactly* `assets` of the underlying token and revert if that is not possible.
- `mint/shares, receiver` must mint *exactly* `shares` vault shares and revert if that is not possible.

The current implementation clamps the requested amount to the remaining cap and silently executes a partial fill:

```
uint256 assetsClamped = assets > maxAllowed ? maxAllowed : assets;
```

This means a caller who requests `deposit(1_000_000e6, receiver)` may only have `300_000e6` pulled from their wallet and only `300_000e6` worth of shares minted with no revert and no indication that the full amount was not deposited. The same applies to `mint`. This breaks the ERC-4626 invariants that integrating protocols, aggregators, and front-ends rely on to know how much was actually deposited.

Recommendation: Remove the clamping logic. ERC-4626 explicitly anticipates that callers should consult `maxDeposit / maxMint` before calling and that the functions revert when the limit is exceeded. Callers are responsible for reading the cap before depositing.

```
- uint256 assetsClamped = assets > maxAllowed ? maxAllowed : assets;  
+ if (assets > maxAllowed) revert ExceedsMaxCap();
```

If the partial-fill UX is intentional, document the ERC-4626 deviation explicitly in both NatSpec and the integration guide, and consider exposing a `depositCapped(uint256 assets, address receiver)` returns `(uint256 assetsActuallyDeposited)` helper that makes the partial-fill semantics opt-in and explicit.

Venus Protocol: Acknowledged. The partial-fill behavior is already documented in the NatSpec and will also be included in the project documentation and UI.

Cantina Managed: Acknowledged.

3.1.4 Defaulting institution can reclaim confiscated margin through supplier shares

Severity: Medium Risk

Context: (No context files were provided by the reviewer)

Description: When a vault fails after raising at least `minBorrowCap`, the institution's margin is intended to be confiscated and distributed to lenders as compensation for the institution failing to complete collateralization.

However, supplier deposits are permissionless. The current position holder can also deposit supply assets during `Fundraising`, receive ERC-4626 shares, intentionally let the vault fail by not topping up to `idealCollateralAmount`, and then redeem those shares to receive a pro-rata portion of the confiscated margin.

This allows the defaulting institution to recover part, or most, of the margin that was meant to compensate external lenders.

Affected Code: Supplier deposits are open to any address:

```
function deposit(
    uint256 assets,
    address receiver
) public override returns (uint256 shares) {
    VaultState s = _checkAndAdvanceState();
    if (s != VaultState.Fundraising) revert InvalidState();
    uint256 maxAllowed = maxDeposit(receiver);
    uint256 assetsClamped = assets > maxAllowed ? maxAllowed : assets;
    if (assetsClamped == 0) revert ExceedsMaxCap();
    shares = previewDeposit(assetsClamped);
    _deposit(_msgSender(), receiver, assetsClamped, shares);
}
```

Failed Scenario B records confiscated margin:

```
if (totalRaised >= _config.minBorrowCap) {
    uint256 marginAmount = (_instConfig.idealCollateralAmount * _instConfig.marginRate)
        ↪ / MANTISSA_ONE;
    _instRuntime.institutionDefaulted = true;
    _instRuntime.confiscatedMarginRemaining = marginAmount;
    emit MarginConfiscated(marginAmount);
}
```

The payout logic distributes confiscated margin only by share ownership:

```
uint256 totalSharesBeforeBurn = totalSupply() + shares;
uint256 compensation = (_instRuntime.confiscatedMarginRemaining * shares) /
    ↪ totalSharesBeforeBurn;

if (compensation > 0) {
    _instRuntime.confiscatedMarginRemaining -= compensation;
    _releaseCollateral(receiver, compensation);
    emit MarginCompensationClaimed(receiver, compensation);
}
```

There is no exclusion for shares held by the defaulting position holder.

Proof of Concept:

1. The institution deposits the required upfront margin.
2. The controller opens the vault into `Fundraising`.
3. The institution also deposits supply assets into the vault and receives supplier shares.
4. External lenders deposit the remaining amount needed to reach `minBorrowCap`.
5. The institution does not deposit enough collateral to reach `idealCollateralAmount`.
6. Fundraising ends and the vault enters `Failed` with `institutionDefaulted = true`.
7. The institution redeems its supplier shares.
8. `_afterWithdrawHook()` pays the institution a pro-rata share of `confiscatedMarginRemaining`.

Recommendation: Possible approaches:

- Disallow the current position holder from calling `deposit()` or `mint()` during `Fundraising`.
- Disallow `receiver == positionHolder` for supplier share minting.
- Track compensation-eligible shares separately and exclude institution-controlled shares from the margin-compensation numerator and denominator.
- Snapshot the defaulting position holder at failure time, but do not rely only on checking the current redeemer address because shares are transferable.

Venus Protocol: Acknowledged. This behavior is acceptable as intended — if an institution chooses to supply assets during `Fundraising` to reduce its own loss exposure, that is within the current design. Furthermore, this scenario is only possible once `minBorrowCap` has already been reached, meaning external suppliers are also participating and would receive their proportional share of the confiscated margin alongside the institution.

Cantina Managed: Acknowledged.

3.1.5 `_baseURI` not being overridden returns empty metadata

Severity: Medium Risk

Context: (No context files were provided by the reviewer)

Description: `InstitutionPositionToken` inherits from OpenZeppelin's `ERC721` but does not override `_baseURI()`. The default implementation returns an empty string, so `tokenURI(uint256 tokenId)` returns an empty string for every token. Every off-chain components that calls `tokenURI` to display the institution's NFT position will show a blank entry with no image, name, or attributes. Having no readable metadata makes it invisible and unusable in any standard NFT interface.

Recommendation: Override `_baseURI()` to point to a hosted metadata endpoint:

```
function _baseURI() internal pure override returns (string memory) {  
    return "https://nftinfo.venus.io/institutional-vault/";  
}
```

Alternatively, implement on-chain SVG metadata following the pattern used by `UniswapV3` or `SablierV2`. Even a minimal static SVG that displays the token ID and vault address is significantly better than returning nothing. On-chain metadata is also more robust as it has no dependency on Venus-hosted infrastructure.

Venus Protocol: Fixed in commit `1f94eba3`.

Cantina Managed: Fix verified.

3.2 Low Risk

3.2.1 `sweep()` can transfer core vault assets without token restrictions

Severity: Low Risk

Context: (No context files were provided by the reviewer)

Description: `BaseVault.sweep(address token)` allows the controller to transfer the entire ERC20 balance of any token held by the vault to the controller treasury. The function does not block core vault tokens such as the supply asset, collateral asset, or the vault share token.

Although the function is restricted by `onlyController`, this creates a governance/configuration footgun. If governance, ACM permissions, controller ownership, or operational scripts are compromised or misused, important vault assets can be swept instead of only recovering unrelated stuck tokens.

Affected Code:

```
function sweep(  
    address token  
) external onlyController {  
    uint256 amount = IERC20(token).balanceOf(address(this));  
    if (amount == 0) revert NothingToSweep();  
    address recipient = IVaultController(vaultController).treasury();  
    IERC20(token).safeTransfer(recipient, amount);  
    emit TokensSwept(token, recipient, amount);  
}
```

Recommendation: Add explicit restrictions for core vault assets, for example:

```
if (token == asset()) revert CannotSweepAsset();  
if (token == address(_instConfig.collateralAsset)) revert CannotSweepCollateral();  
if (token == address(this)) revert CannotSweepShares();
```

Venus Protocol: Acknowledged. The function is controlled by governance through either a timelock or a multisig guardian. The lack of additional guards is intentional — flexibility is needed for unexpected scenarios, especially since the contract is non-upgradeable.

Cantina Managed: Acknowledged.

3.2.2 PSR `updateAssetsState` failure is handled inconsistently

Severity: Low Risk

Context: (No context files were provided by the reviewer)

Description: The two call sites that notify the Protocol Share Reserve (PSR) after transferring tokens to it handle failure in opposite ways with no documented rationale.

In `BaseVault._settleProtocolShare`, the `updateAssetsState` call is wrapped in a `try/catch` that silently discards any revert.

In `LiquidationAdapter`, the equivalent PSR notification is called without any `try/catch`, so a revert there propagates and causes the entire liquidation to revert.

Note: A review of the PSR contract confirms that `updateAssetsState` has no reason to revert as long as the comptroller and asset addresses are correctly registered in the system.

Recommendation: Apply a single policy to both call sites. Given that the PSR contract cannot fail under correct configuration, treating a revert as a misconfiguration error and letting it propagate is consistent with how `LiquidationAdapter` already behaves.

Venus Protocol: Acknowledged. The `try/catch` in `BaseVault._settleProtocolShare` is intentional — it follows the existing Core Protocol logic where `updateAssetsState` calls are wrapped because the operation itself should not be blocked by a PSR state update failure. In `LiquidationAdapter`, however, PSR transfers are direct and explicit, so any failure there should correctly cause a revert.

Cantina Managed: Acknowledged.

3.2.3 Close-factor rounding can block liquidation of dust debt

Severity: Low Risk

Context: (No context files were provided by the reviewer)

Description: Liquidation repayment is capped by the configured close factor:

```
uint256 maxRepay = (debt * closeFactor) / MANTISSA_ONE;  
if (actualRepay > maxRepay) revert ExceedsCloseFactor();
```

When `closeFactor < 1e18` and the remaining debt is very small, this calculation can round `maxRepay` down to zero. For example, with `debt = 1` and `closeFactor = 0.5e18`, `maxRepay = 0`.

Since liquidation requires a non-zero repayment, both `liquidate()` and `liquidateOverdueVault()` can become unable to clear the final dust amount.

Impact: The vault can remain in `PendingSettlement` or `SettlementDeadlineExceeded` with a tiny non-zero debt that liquidation paths cannot repay due to close-factor rounding.

This can temporarily block supplier redemption because the vault only matures once debt reaches zero. However, the issue is low severity because `repay()` is permissionless and can clear the dust directly:

```
function repay(
    uint256 amount
) external nonReentrant whenNotCompletelyPaused {
    _repay(msg.sender, amount);
}
```

So any user, keeper, or lender can repay the final unit and allow the vault to transition normally.

Proof of Concept:

1. The institution repays almost all outstanding debt, leaving `1` smallest unit.
2. The vault is in `PendingSettlement` or `SettlementDeadlineExceeded`.
3. A liquidator or overdue settler attempts to repay the final unit.
4. `maxRepay = debt * closeFactor / 1e18` rounds to `0`.
5. The liquidation reverts with `ExceedsCloseFactor`.
6. A normal `repay(1)` call is still required to clear the dust and let the vault mature.

Recommendation: Ensure that liquidation can always repay at least one unit when debt is non-zero:

```
uint256 maxRepay = (debt * closeFactor) / MANTISSA_ONE;
if (maxRepay == 0 && debt > 0) {
    maxRepay = 1;
}
```

Alternatively, bypass the close-factor cap when `debt` is below a configured dust threshold.

Venus Protocol: Acknowledged. It is extremely unlikely that liquidators would attempt liquidation to repay a dust amount such as `1` unit. The remaining debt can be cleared through a normal `repay()` call, so the current behavior is acceptable.

Cantina Managed: Acknowledged.

3.2.4 Inconsistent handling of Fee-on-Transfer tokens

Severity: Low Risk

Context: (No context files were provided by the reviewer)

Description: `depositCollateral` records the actual received amount using a before/after balance difference:

```
uint256 balanceBefore = collateralToken.balanceOf(address(this));
collateralToken.safeTransferFrom(msg.sender, address(this), amount);
uint256 actual = collateralToken.balanceOf(address(this)) - balanceBefore;
```

No other transfer site in the codebase uses this pattern without a reason. Supply token transfers in `_receiveRepayment`, `_receiveDeposit`, and collateral transfers in liquidation all use direct `safeTransferFrom` without recording the actual received amount.

This inconsistency implies either: (a) fee-on-transfer tokens were once in scope and the pattern was applied incompletely, or (b) the pattern is defensive boilerplate applied to collateral only, creating a misleading signal that fee-on-transfer tokens are supported when they are not.

If fee-on-transfer tokens are out of scope, the balance-diff pattern in `depositCollateral` should be removed so all transfer sites are uniform. If fee-on-transfer tokens are in scope for collateral, the same pattern must be applied everywhere a token amount is recorded.

Recommendation: Since fee-on-transfer tokens are confirmed to be out of scope, remove the balance-diff pattern and use direct amount recording:

```
- uint256 balanceBefore = collateralToken.balanceOf(address(this));
+ collateralToken.safeTransferFrom(msg.sender, address(this), amount);
- uint256 actual = collateralToken.balanceOf(address(this)) - balanceBefore;
```

This removes the false implication that fee-on-transfer tokens are supported, eliminates a redundant external call, and makes the codebase uniform.

Venus Protocol: Fixed in commit 58b0e1c5.

Cantina Managed: The fix retains the existing balance-diff pattern in `depositCollateral` (which has been audited twice) and instead adds NatSpec comments explicitly clarifying that fee-on-transfer tokens are not supported, making the intent of the pattern clear without restructuring the code.

3.3 Gas Optimization

3.3.1 Dead code: `positionTokenId == 0` guard can never be triggered

Severity: Gas Optimization

Context: (No context files were provided by the reviewer)

Description: `initialize` contains the following guard:

```
if (_instConfig.positionTokenId == 0) revert PositionTokenIdNotSet();
```

`InstitutionPositionToken` initialises its internal counter at 1, and `mint` returns `nextTokenId` before incrementing. Every vault created through `createVault()` receives a `positionTokenId >= 1`. The revert condition `positionTokenId == 0` is therefore an invariant violation that can only be reached by initialising a vault clone directly with a hand-crafted config that bypasses the controller and that path is out of scope.

The guard costs gas on every legitimate vault initialization and gives the false impression that `positionTokenId = 0` is a plausible runtime condition worth defending against.

Recommendation: Remove the dead check and also remove the `PositionTokenIdNotSet` error definition since it has no other callers.

Venus Protocol: Fixed in commit a756a8d6.

Cantina Managed: Fix verified.

3.3.2 Unnecessary address prediction in `createVault`

Severity: Gas Optimization

Context: (No context files were provided by the reviewer)

Description: `createVault` currently runs in three steps: predict the deterministic clone address, mint the NFT with that predicted address as the recipient, then deploy the clone:

```
vault = Clones.predictDeterministicAddress(vaultImplementation, salt);
uint256 tokenId = positionToken.mint(institution, vault);

// Deploy clone
vault = Clones.cloneDeterministic(vaultImplementation, salt);
```

The `predictDeterministicAddress` call costs gas and is only needed so the vault address is known before the NFT is minted. However, the NFT mint recipient is the vault address only because the vault initialization needs the `tokenId`, not because the vault needs to hold the NFT. The vault's `initialize` receives `tokenId` as a parameter. There is no circular dependency that requires the address prediction.

Recommendation: Deploying the clone first and then minting the NFT with the real deployed address eliminates the prediction call entirely:

```
- vault = Clones.predictDeterministicAddress(vaultImplementation, salt);  
- uint256 tokenId = positionToken.mint(institution, vault);  
  
// Deploy clone  
vault = Clones.cloneDeterministic(vaultImplementation, salt);  
+ uint256 tokenId = positionToken.mint(institution, vault);
```

Venus Protocol: Fixed in commit [a756a8d6](#).

Cantina Managed: Fix verified.

3.4 Informational

3.4.1 NatSpec for `openVault` references vault state "Open" that doesn't exist

Severity: Informational

Context: (No context files were provided by the reviewer)

Description: The NatSpec comment for `openVault` in `InstitutionalLoanVault` reads:

```
@notice Transitions MarginDeposited -> Open. Controller only.
```

There is no vault state named `Open` in the `VaultState` enum. The correct target state is `Fundraising`. The same incorrect state name appears in at least two other locations in the codebase. This creates confusion for reviewers and integrators trying to map comments to actual on-chain behaviour.

Recommendation: Update all three occurrences:

```
- * @notice Transitions MarginDeposited -> Open. Controller only.  
+ * @notice Transitions MarginDeposited -> Fundraising. Controller only.
```

Search the codebase for `-> Open` and `Open state` and replace with the correct state name `Fundraising`.

Venus Protocol: Fixed in commit [455cb442](#).

Cantina Managed: Fix verified.

3.4.2 NatSpec for `_enterLock` claims a USD valuation is stored when it is not

Severity: Informational

Context: (No context files were provided by the reviewer)

Description: The NatSpec comment for `_enterLock` reads:

```
@dev Fundraising -> Lock. Initialises totalDebt, computes and stores minimum collateral  
required and its USD valuation scaled to the actual amount raised.
```

`_enterLock` computes and stores `minimumCollateralRequired` which is a collateral-native unit value. No USD valuation is computed or stored anywhere in the function. The phrase "and its USD valuation" is inaccurate.

Recommendation: Update the comment as suggested.

Venus Protocol: Fixed in commit [455cb442](#).

Cantina Managed: Fix verified.

3.4.3 `withdraw` and `redeem` duplicates ERC-4626 Logic

Severity: Informational

Context: (No context files were provided by the reviewer)

Description: Both `withdraw` and `redeem` re-implement the cap check and the `_withdraw` dispatch that OpenZeppelin's ERC-4626 base already performs. The only additions are the state gate check and the `ExceedsMaxCap` error name. The rest of the function body is a verbatim reimplement of the parent logic:

```
uint256 maxAssets = maxWithdraw(owner);
if (assets > maxAssets) revert ExceedsMaxCap();
shares = previewWithdraw(assets);
 Withdraw(_msgSender(), receiver, owner, assets, shares);
```

Recommendation: Delegate to `super` after the state gate:

```
function withdraw(
    uint256 assets,
    address receiver,
    address owner
) public override returns (uint256 shares) {
    VaultState s = _checkAndAdvanceState();
    if (s != VaultState.Matured && s != VaultState.Failed && s != VaultState.Liquidated)
        ↪ revert InvalidState();
    return super.withdraw(assets, receiver, owner);
}

function redeem(
    uint256 shares,
    address receiver,
    address owner
) public override returns (uint256 assets) {
    VaultState s = _checkAndAdvanceState();
    if (s != VaultState.Matured && s != VaultState.Failed && s != VaultState.Liquidated)
        ↪ revert InvalidState();
    return super.redeem(shares, receiver, owner);
}
```

The OZ base already calls `maxWithdraw` / `maxRedeem` and reverts with the standard ERC-4626 message on overflow.

Venus Protocol: Acknowledged. The refactor is correct in principle, but a change to already-audited code is not preferred at this stage. Custom errors are gas-efficient, and the current structure makes internal flow easier to follow without requiring readers to trace into base class implementations.

Cantina Managed: Acknowledged.

3.4.4 Oracle pricing logic is duplicated across multiple functions

Severity: Informational

Context: (No context files were provided by the reviewer)

Description: The pattern of fetching the oracle reference, calling `getPrice`, and computing `amount * price / MANTISSA_ONE` appears independently in `_getCollateralValueUSD`, `_getDebtValueUSD`, and `_calculateSeizeAmount`. Any future change to pricing logic must be applied in three places and is prone to partial application.

Recommendation: Extract a single shared helper and have the three functions delegate to it:

```

function _getAssetValueUSD(address assetAddr, uint256 amount) internal view returns
↳ (uint256) {
    if (amount == 0) return 0;
    IResilientOracle oracleRef =
    ↳ IResilientOracle(IInstitutionalVaultController(vaultController).oracle());
    uint256 price = oracleRef.getPrice(assetAddr);
    if (price == 0) revert InvalidOraclePrice();
    return (amount * price) / MANTISSA_ONE;
}

function _getCollateralValueUSD(uint256 amount) internal view returns (uint256) {
    return _getAssetValueUSD(address(_instConfig.collateralAsset), amount);
}

function _getDebtValueUSD(uint256 amount) internal view returns (uint256) {
    return _getAssetValueUSD(asset(), amount);
}

```

Refactor `_calculateSeizeAmount` to use the same helper in place of its inline price lookups as well.

Venus Protocol: Fixed in commit [38393a64](#), with an additional refactor in commit [b62cc195](#).

Cantina Managed: Fix verified.